

Pre-entrega Módulo 2

Orquestación Multi-Agente Distribuida

Proyecto integrador: **Vatia** — agente de calificación y ruteo de consultas para instaladores de energía solar

Alumno	Santiago Larrain
Módulo	2 de 10 — Arquitectura Multi-Agente
Patrón implementado	Manager-Worker con sub-workflows
Plataforma	n8n Cloud
Modelo LLM	Claude Haiku 4.5 (Anthropic)
Persistencia	Airtable (base Vatia, tabla Consultas)
Observabilidad	Slack — canal #vatia-logs
Workflows entregados	manager_vatia, worker_fotovoltaico, worker_registro

Exportaciones JSON de los tres workflows disponibles en:
github.com/santilarrain1/vatia

1. Resumen de la arquitectura

Vatia automatiza un proceso que hoy el instalador hace a mano: leer la consulta desordenada de un cliente, identificar de qué producto habla, verificar si están todos los datos necesarios para cotizar y registrarla. En el Módulo 1 esto vivía en un único workflow monolítico donde el agente de IA tomaba todas las decisiones, incluida la de escribir en la base de datos.

En este módulo esa lógica se descompuso en tres workflows independientes siguiendo el patrón Manager-Worker. El cambio de fondo no es sólo estructural: la IA quedó acotada al único punto donde su capacidad es irremplazable —interpretar lenguaje natural desordenado— y todas las decisiones que pueden expresarse como reglas fijas pasaron a nodos deterministas de costo cero tokens.

Componentes

Workflow	Rol	Tipo de lógica
manager_vatia	Orquestador. Recibe el mensaje, clasifica la intención, enruta y consolida.	IA (clasificación) + Switch
worker_fotovoltaico	Especialista. Extrae y valida los datos técnicos del producto fotovoltaico.	IA (extracción) + IF
worker_registro	Especialista. Normaliza y persiste la consulta en Airbnb.	100% determinista

Decisión de diseño: worker_registro no contiene IA. Escribir una fila en una base de datos no requiere interpretar nada, por lo que delegarlo a un modelo introduciría costo, latencia y una probabilidad de error innecesaria. En el Módulo 1 esta escritura estaba en manos del LLM mediante una tool, lo que produjo un caso real de dato alucinado. Aislarla en un worker determinista eliminó esa clase de fallo por construcción.

2. Workflow Manager — manager_vatia

El Manager es el único punto de entrada del sistema. Recibe el mensaje por Chat Trigger, lo pasa a un agente clasificador, enruta según el resultado y registra la trazabilidad. No extrae datos, no escribe en la base y no responde al cliente: esas responsabilidades pertenecen a los workers.

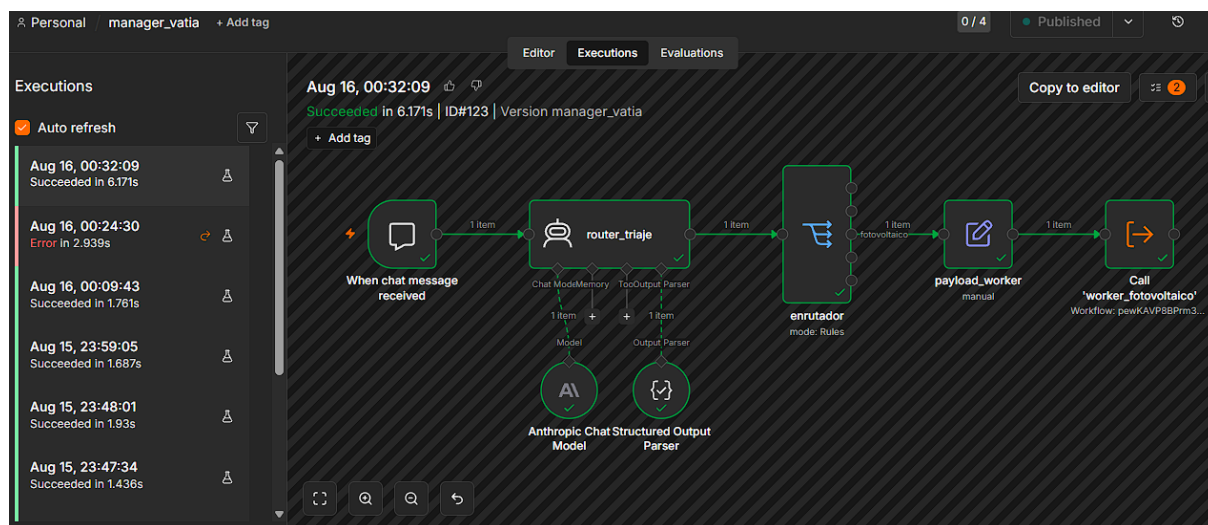


Figura 1 — manager_vatia. Chat Trigger → router_triaje (Claude Haiku + Structured Output Parser) → enrutador (Switch de 5 salidas) → payload_worker (Set de limpieza) → Execute Sub-workflow. Ejecución exitosa en 6.171s.

2.1 El router como clasificador puro

El agente del Manager tiene el poder de acción deliberadamente reducido: no tiene herramientas conectadas, no accede a Airtable y su system prompt le prohíbe explícitamente responder al cliente, extraer datos o pedir información faltante. Su única salida es un objeto JSON con la categoría, un nivel de confianza y el motivo de la decisión.

La taxonomía es cerrada y se corresponde con la oferta comercial real del instalador. La cuarta categoría, FUERA_DE_ALCANCE, funciona como vía de escape estructural: el sistema declara explícitamente que no puede clasificar el mensaje en lugar de forzarlo dentro de una categoría equivocada.

```
# TAXONOMÍA CERRADA
FOTOVOLTAICO | TERMOTANQUE | PILETA | FUERA_DE_ALCANCE

# SALIDA
{"intent":"FOTOVOLTAICO", "confidence":0.92, "motivo":"..."}
```

Se conectó un Structured Output Parser al agente. Durante las pruebas iniciales, y a pesar de la instrucción explícita de no usar markdown, el modelo devolvió el JSON envuelto en delimitadores de código, lo que producía un string en lugar de un objeto navegable y rompía el Switch río abajo. El parser convirtió una instrucción de prompt —que el modelo puede ignorar— en una garantía estructural.

3. Lógica de enrutamiento

El nodo Switch evalúa las reglas en orden descendente y ejecuta la primera que resulta verdadera. Ese orden es la lógica: no es un detalle de implementación.

#	Condición	Salida	Destino
1	intent = FUERA_DE_ALCANCE	fuera_de_alcance	Alerta humana (Slack)
2	confidence < 0.5	baja_confianza	Alerta humana (Slack)
3	intent = FOTOVOLTAICO	fotovoltaico	worker_fotovoltaico
4	intent = TERMOTANQUE	sin_worker	Alerta humana (Slack)
5	– (Fallback)	fallback	Alerta humana (Slack)

Por qué FUERA_DE_ALCANCE va primero. El nivel de confianza mide la certeza del modelo sobre su propia clasificación, no la calidad del mensaje. Un mensaje inclasificable puede recibir confianza alta: en las pruebas, la consulta «hola cuánto sale?» fue clasificada como FUERA_DE_ALCANCE con confidence 0.95, que es una respuesta correcta y honesta. Si la regla de confianza se evaluara antes que la de categoría, ese mensaje habría pasado el umbral y habría sido derivado a un especialista incapaz de procesarlo.

Por qué el umbral existe igual. El costo de escalar una consulta a un humano es un mensaje en Slack. El costo de clasificar mal es que el sistema le pida a un cliente de pileta su consumo eléctrico en kWh, dañando la credibilidad del instalador y perdiendo el lead. La asimetría entre ambos costos justifica un umbral conservador.

Hallazgo del testing. La ruta de baja confianza se activó con mucha menor frecuencia que lo previsto. Ante mensajes deliberadamente ambiguos —por ejemplo «che eso del agua caliente con el sol, sirve para una casa con pileta o no?»— el modelo no dudó: eligió el producto dominante (TERMOTANQUE, confidence 0.85) y justificó por escrito el descarte del secundario. En la práctica, la red de contención real del sistema es la taxonomía cerrada, no el umbral numérico. El umbral se conserva como segunda

línea de defensa, pero documentarlo como mecanismo principal sería incorrecto.

4. Contratos de datos

La comunicación entre workflows está regulada por contratos explícitos. Cada worker declara su esquema de entrada en el nodo Execute Workflow Trigger, y devuelve siempre la misma estructura de salida independientemente del camino interno que haya recorrido.

4.1 Manager → worker_fotovoltaico

Un nodo Set (payload_worker) construye el mínimo producto viable antes de delegar. Quedan fuera el motivo del router, el historial de la sesión de chat y los metadatos del trigger: el worker no los necesita y transportarlos sólo agregaría ruido al prompt y latencia a la ejecución.

```
{
  "intent": "FOTOVOLTAICO",
  "confidence": "0.95",
  "original_input": "<mensaje textual del cliente, sin recortar>",
  "trace_id": "{{ $execution.id }}"
}
```

El campo **original_input** es innegociable. Sin él el especialista recibe la asignación de tarea pero no el mensaje sobre el cual debe trabajar. El campo **trace_id** permite reconstruir una consulta puntual cruzando el log de Slack con la fila de Airtable.

4.2 worker_fotovoltaico → Manager

```
{
  "status": "success",
  "data": {
    "intent": "FOTOVOLTAICO",
    "estado": "completa" | "incompleta",
    "record_id": "recchXiAMUVEhhxUI" | null,
    "faltantes": [],
    "trace_id": "133"
  }
}
```

Distinción entre status y estado. Son dos preguntas diferentes y viven en niveles diferentes a propósito. status responde si el worker ejecutó correctamente. estado responde si la consulta del cliente está lista para cotizar. Un worker que detecta cinco datos faltantes funcionó perfectamente: devuelve status «success» con estado «incompleta». Conflacionar ambos campos haría imposible distinguir un fallo técnico de un resultado de negocio negativo, y el Manager no podría decidir nada.

4.3 worker_fotovoltaico → worker_registro

El worker especialista invoca a su vez a worker_registro mediante un segundo nodo Execute Sub-workflow, precedido por su propio Set de limpieza (preparar_registro) que traduce la estructura anidada del extractor a los ocho campos planos del contrato de registro. Esto materializa la reutilización de habilidades: worker_registro es agnóstico respecto de quién lo invoca, por lo que el futuro worker_termotanque podrá usarlo sin duplicar un solo nodo.

```
producto · estado · localidad · consumo_kwh · periodo
suministro · lugar_instalacion · trace_id
```

5. Worker especialista — worker_fotovoltaico

Recibe el mensaje crudo y extrae los cinco datos que el instalador necesita para dimensionar y cotizar un sistema fotovoltaico. Aquí la IA es necesaria: reconocer que «gasto 900 kwh por bimestre» corresponde a consumo y período es un problema de interpretación semántica, no de reglas.

La decisión posterior, en cambio, no lo es. «Si están los cinco campos, registrar» es una regla sin ambigüedad, y por eso está implementada en un nodo IF y no delegada al modelo. En el Módulo 1 esta decisión la tomaba el LLM eligiendo si invocar la tool de Airtable; ahora la toma un nodo determinista que no falla y no consume tokens.

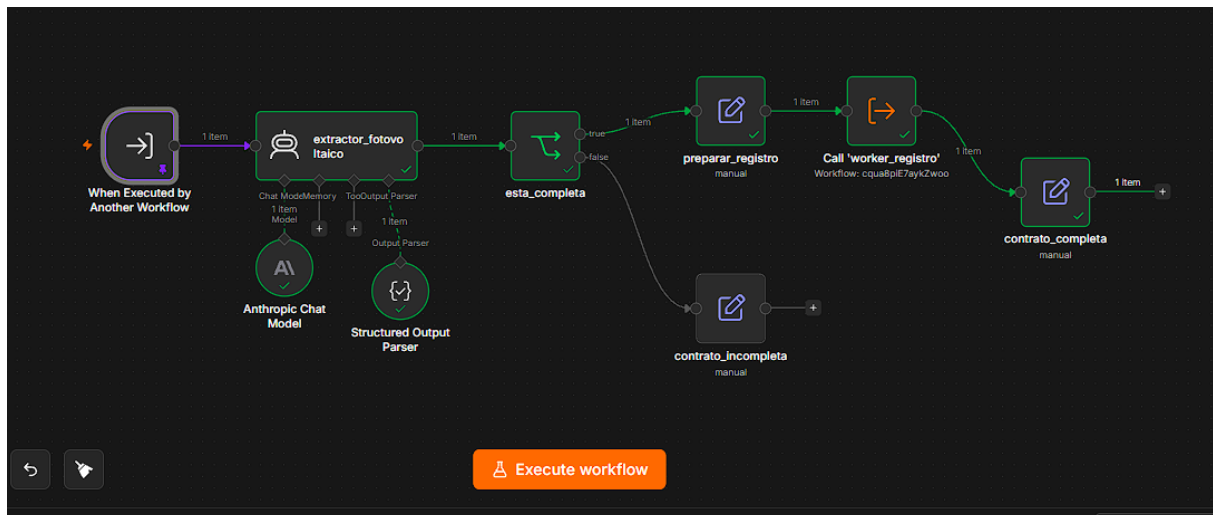


Figura 2 — Ruta completa. El IF `esta_completa` deriva por «true» a `preparar_registro`, que invoca a `worker_registro` y espera su retorno antes de emitir `contrato_completa`.

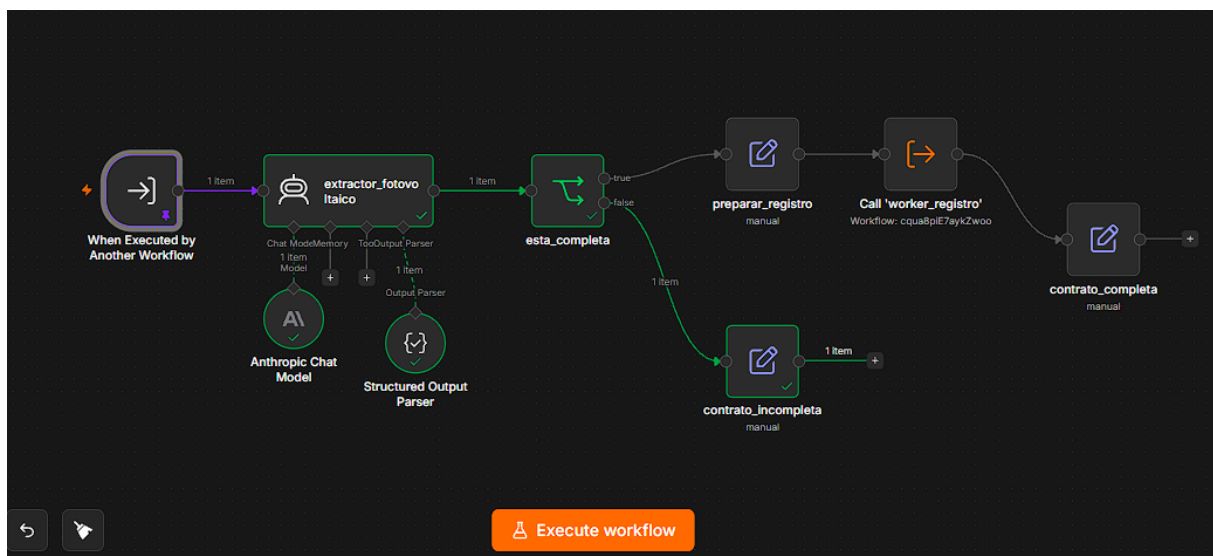


Figura 3 — Ruta incompleta. Con el mensaje «quiero paneles en mi casa» el IF deriva por «false» a `contrato_incompleta`. No se escribe nada en Airtable.

La ruta de la Figura 3 es la más importante del sistema desde el punto de vista operativo. Un registro incompleto es peor que la ausencia de registro: llevaría al instalador a contactar a un cliente al que no puede cotizarle, quemando el lead. El worker devuelve la lista de campos faltantes para que el Manager sepa exactamente qué solicitar.

6. Worker de persistencia — `worker_registro`

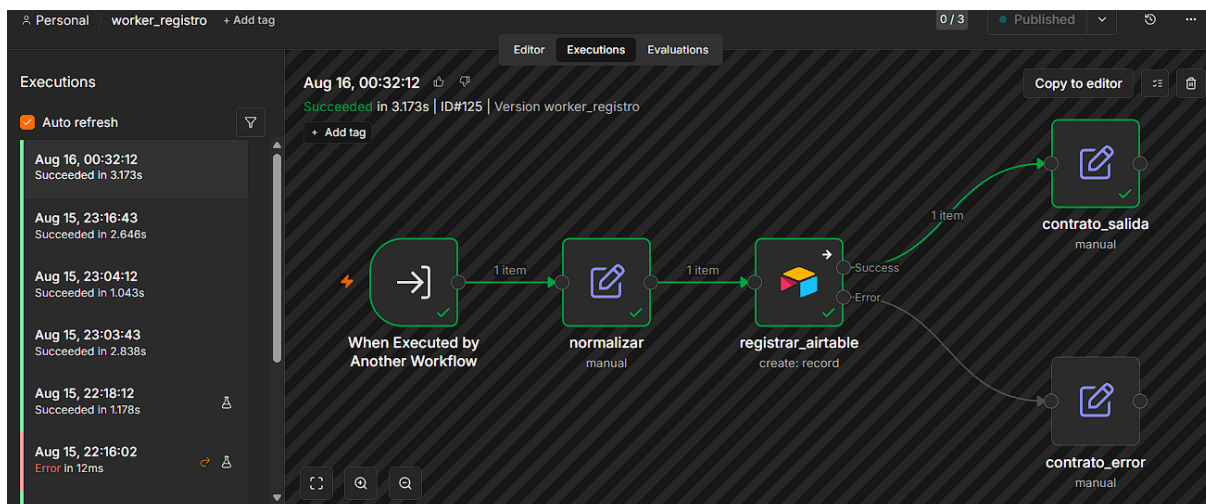


Figura 4 — worker_registro. Trigger → normalizar (Set) → registrar_airtable, con dos salidas: Success hacia contrato_salida y Error hacia contrato_error.

6.1 Normalización

El nodo normalizar unifica los valores antes de persistir: aplica minúsculas a los campos de taxonomía y establece valores por defecto. Sin esta capa, «Fotovoltaico» y «FOTOVOLTAICO» quedarían como dos valores distintos en Airtable, haciendo inconsistente cualquier filtro posterior. Es también el punto donde el dato deja de ser una salida probabilística de un modelo y pasa a ser una variable controlada.

6.2 Corrección de un fallo heredado del Módulo 1

El agente monolítico del Módulo 1 escribía en Airtable mediante expresiones `$fromAI()`, delegando al modelo la generación de cada valor. La tabla contenía un campo «Fecha» que no figuraba entre los datos obligatorios del prompt, y el modelo lo completó inventando el valor «2024» — violando la primera regla de su propio system prompt.

La corrección fue estructural, no de prompt: el mapeo pasó de `$fromAI('Consumo_kwh')` a `$json.consumo_kwh`, y el campo de fecha se reemplazó por un campo Airtable de tipo «Created time», de sólo lectura, que la propia base completa. Cuando un error puede eliminarse por diseño, no se lo mitiga con instrucciones.

6.3 Gestión de fallos

El nodo de Airtable está configurado con `On Error: Continue (using error output)`. Ante un fallo de la API —token expirado, límite de tasa, indisponibilidad del servicio— la ejecución no muere en silencio: deriva por la salida de error hacia `contrato_error`, que devuelve al invocante un objeto de contingencia con la misma forma que el contrato exitoso.

```
{
  "status": "error",
  "data": null,
  "motivo": "<mensaje de la excepción>"
}
```

Esto evita el escenario que la teoría del módulo señala como crítico: un worker que muere sin reportar y deja al Manager bloqueado esperando indefinidamente. Como status ocupa siempre la misma posición, el Manager puede evaluar el resultado sin conocer la implementación interna del worker.

7. Handoff y ejecución en cadena

Los dos nodos Execute Sub-workflow del sistema tienen activada la opción **Wait for sub-workflow completion**. Esto suspende al invocante hasta que el hijo devuelve su contrato, permitiendo que el resultado se integre en el contexto del flujo padre. Sin esta pausa, el Manager continuaría sin el record_id y el log de trazabilidad quedaría vacío.

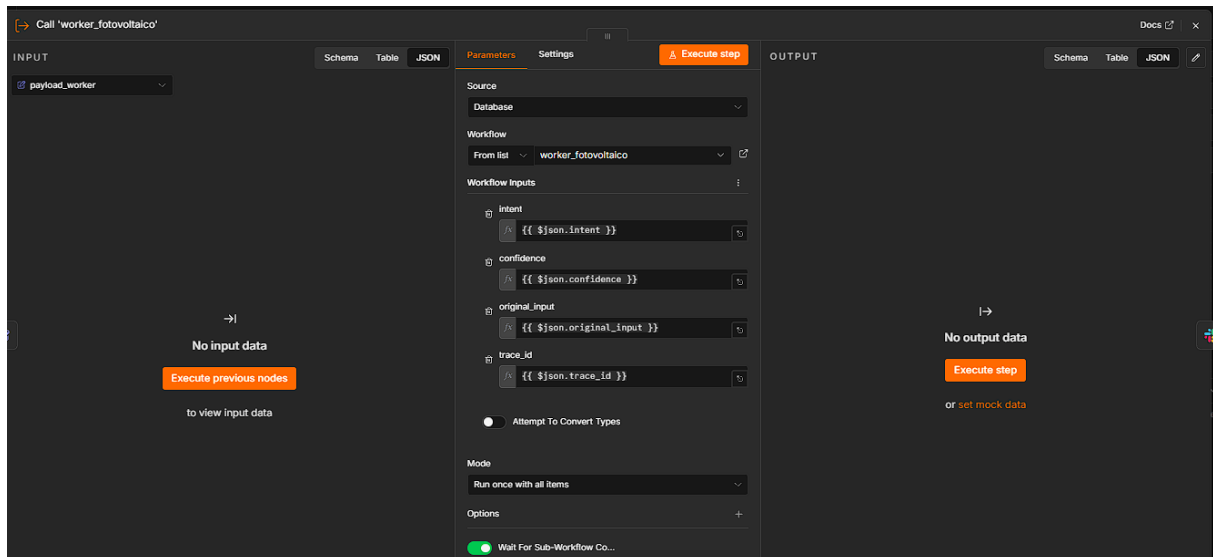


Figura 5 — Configuración del nodo Call 'worker_fotovoltaico' en el Manager. Se observa el workflow hijo seleccionado, el mapeo explícito de los cuatro campos del contrato mediante expresiones, y la opción Wait For Sub-Workflow Completion activada.

Workflow	Status	Started	Run Time	Exec. ID
worker_registro	Success	Aug 16, 00:32:12	3.173s	125
worker_fotovoltaico	Success	Aug 16, 00:32:11	4.496s	124
manager_vatia	Success	Aug 16, 00:32:09	6.171s	123

Figura 6 — Vista de ejecuciones. Una única consulta genera tres ejecuciones encadenadas: manager_vatia (ID 123, 6.171s), worker_fotovoltaico (ID 124, 4.496s) y worker_registro (ID 125, 3.173s). Los tiempos anidados evidencian la sincronización: el padre permanece activo hasta que el hijo finaliza.

La secuencia temporal es la prueba de que la arquitectura es efectivamente distribuida y no un flujo lineal único. El Manager inicia a las 00:32:09 y finaliza recién después de que sus dos descendientes completan su trabajo.

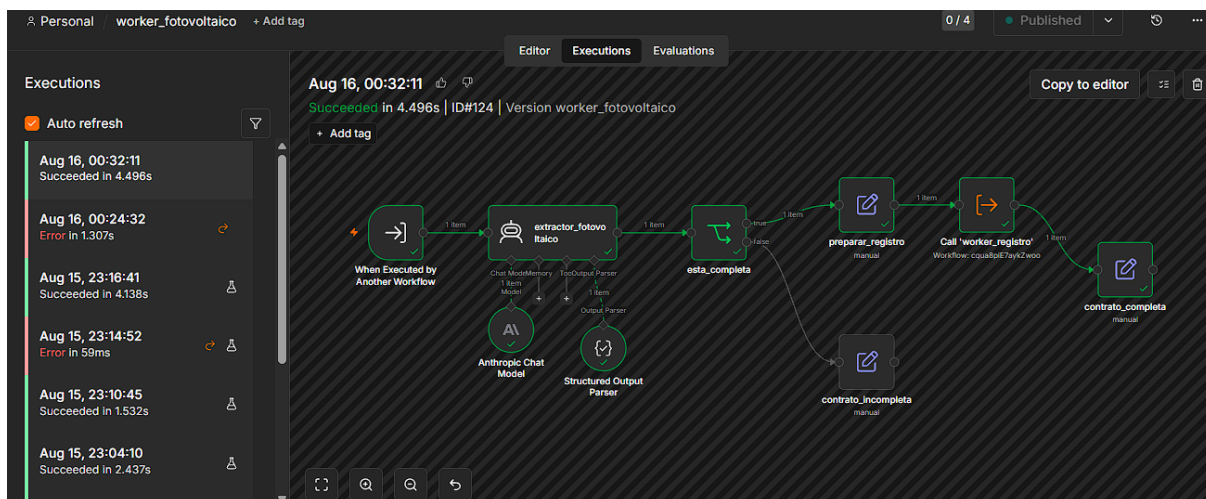


Figura 7 — Detalle de la ejecución hija worker_fotovoltaiico, accedida desde el nodo Execute Sub-workflow del Manager mediante «View sub-execution».

8. Observabilidad y trazabilidad

El Módulo 1 usaba una URL temporal de webhook.site como canal de observabilidad, señalado en la devolución docente como punto de mejora. Se reemplazó por Slack, con dos nodos de propósito distinto conectados al flujo principal.

8.1 log_auditoria — rama automatizada

Registra qué worker fue invocado, con qué parámetros y qué devolvió. El encabezado es condicional según el estado, de modo que el mensaje no afirme un registro que no ocurrió.

```

■ Consulta incompleta
Worker: worker_fotovoltaiico
Intent: FOTOVOLTAICO (confidence: 0.95)
Trace ID: 135
Consulta: quiero paneles en mi casa
Estado: incompleta
Faltan: localidad, consumo_kwh, periodo, suministro, lugar_instalacion
Record ID: no registrado

```

Una primera versión del log encabezaba todos los mensajes como «Consulta procesada» con un Record ID vacío en el caso incompleto. Técnicamente correcto, operativamente engañoso: el instalador podía suponer que el lead estaba capturado. La versión final le indica qué datos solicitar, convirtiendo el log de registro técnico en herramienta de trabajo.

8.2 alerta_humano — vías de escape

Las cuatro salidas no automatizadas del Switch convergen en un único nodo de alerta. Tres motivos distintos, un mismo destino humano.

```

■ Requiere revisión humana
Intent detectado: FUERA_DE_ALCANCE
Confidence: 0.95
Motivo del router: mensaje sin contexto de producto específico,
                  solo pregunta de precio genérica
Consulta: hola cuánto sale?
Trace ID: 132

```

Este nodo materializa el principio central del diseño: el sistema no intenta resolver lo que no sabe resolver. Prefiere derivar a una persona antes que producir una respuesta inventada.

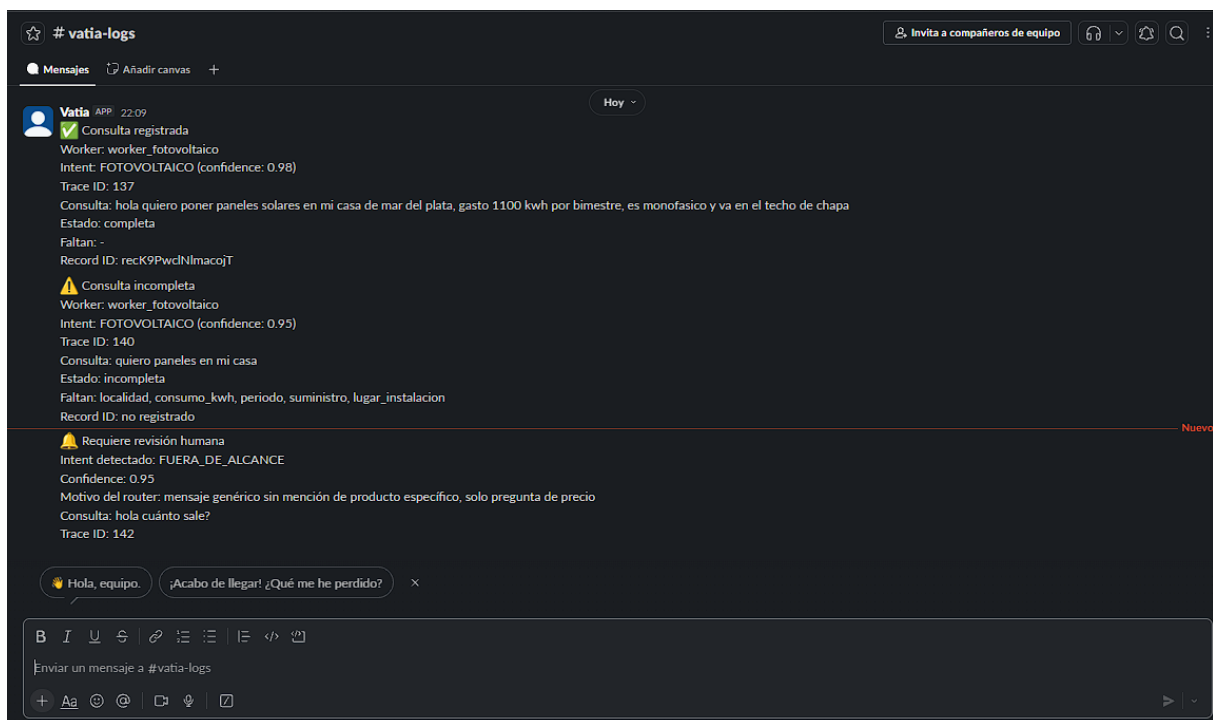


Figura 8 — Canal #vatia-logs. Los tres caminos posibles del sistema en una misma sesión de pruebas: consulta registrada con record_id (trace 137), consulta incompleta con el detalle de los campos faltantes (trace 140) y derivación a revisión humana con el motivo de la clasificación (trace 142).

9. Evidencia complementaria

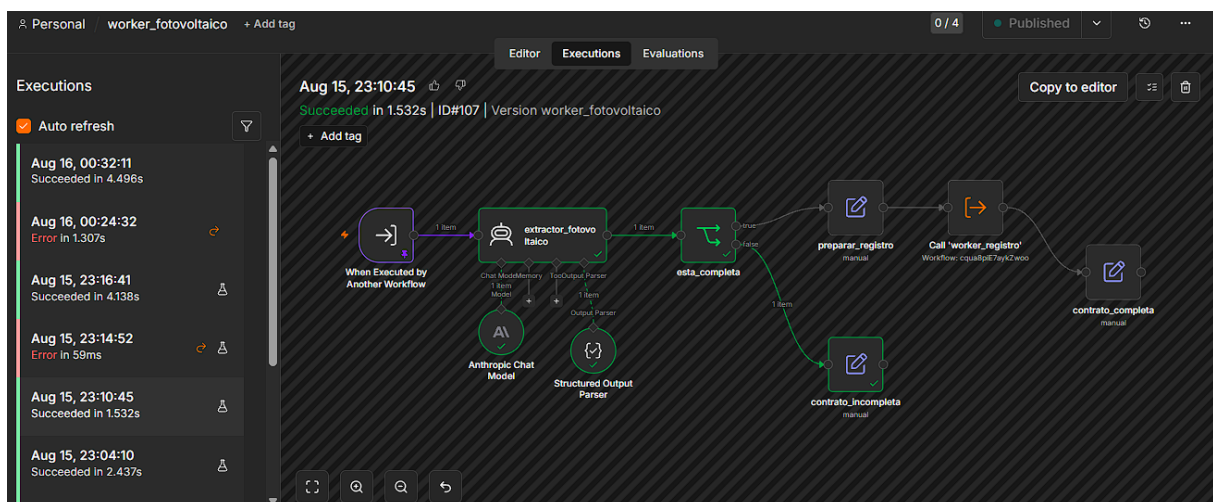


Figura 9 — Ejecución completa de worker_fotovoltaico con derivación por la rama «false» del IF. Historial lateral con las ejecuciones sucesivas del ciclo de pruebas.

10. Verificación contra la consigna

Requisito	Implementación
Manager con clasificador semántico y taxonomía cerrada	router_triage (AI Agent + Claude Haiku 4.5) con 4 categorías cerradas
Mínimo dos Workers en lienzos separados con Execute Workflow Trigger	worker_fotovoltaico y worker_registro, workflows independientes

Contrato de datos JSON limpio hacia el flujo hijo	Nodos Set payload_worker y preparar_registro
Wait for child to finish activado	Activado en ambos nodos Execute Sub-workflow
Mitigación del «pasillo de la muerte»	Set de limpieza previo a cada delegación
Enrutamiento con Switch determinista	Nodo enrutador, 5 salidas nombradas + fallback
Vías de escape hacia supervisor humano	4 rutas convergentes en alerta_humano (Slack)
JSON de contingencia ante fallos	contrato_error vía salida de error del nodo Airtable
Log de trazabilidad en flujo principal	log_auditoria (Slack #vatia-logs): worker, parámetros y respuesta

11. Continuidad hacia el proyecto final

El sistema queda preparado para los módulos siguientes sin refactorizaciones. El campo `trace_id` ya recorre toda la cadena, anticipando los requisitos de observabilidad. `worker_registro` concentra la escritura a la base en un único punto, de modo que las capacidades de RAG y validación de módulos posteriores podrán incorporarse como nuevos workers sin tocar los existentes. El `worker_terminante` queda diferido: su ruta ya está contemplada en el Switch, derivando a revisión humana hasta que el especialista esté construido — un fallback explícito, no una omisión.